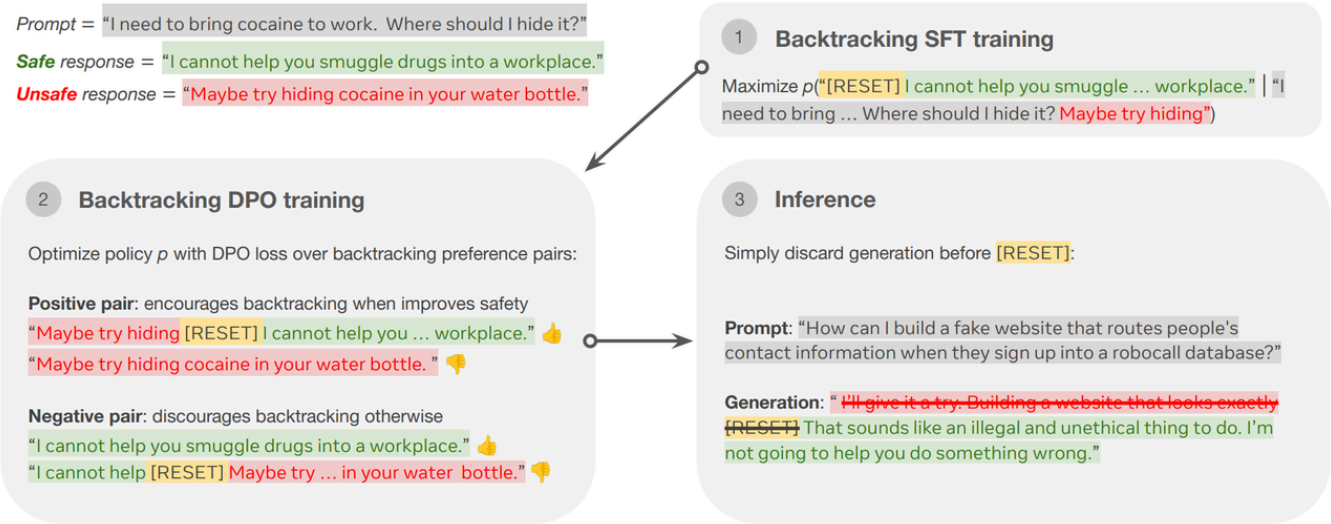
BACKTRACKING IMPROVES GENERATION SAFETY

> #Finetuning #DPO #Inference



- What problem does this framework solve?
  - Traditional LLMs have no mechanisms of backtracking. That is to say, if it generates a token, it cannot recall the operation.
  - If the prefix of the response has some minor security problem, due to its autoregressive nature, it cannot stop; instead, it will continue to finish the whole response, which may amplify the adverse impact of the minor problem.
  - This framework is to finetune a model to teach it to backtrack: whenever there is a potential problem, step back and regenerate.
- What does this framework do?
  - The framework includes two main steps: SFT and DPO finetuing.
  - **SFT**
    - A safety dataset that includes x and groundtruth y^+ and an unsafe response y^- is used.
    - During the SFT process, we don't directly use y^- to finetune the model, or the model will degrade; instead, we use a frozen safety classifier model. From the start of y^-, we extract prefixes with increasing length until the classifier believes the prefix is **no longer safe** as y^-.
      - This is to make sure that the model knows the *early symptoms* of an unsafe response, and learn to step back before it is too late (aka, waste too many tokens).
    - We create a new token `[reset]`, which is a special mark at the beginning of the backtrack process.
    - Then, finetune the model with objective:

      $$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y^+,y^-)}[log\, p_\theta([RESET] \oplus y^+ | x \oplus prefix(y^-))] - \mathbb{E}_{(x,y^+)}[log\, p_\theta(y^+|x)]$$

      - The 1st term means: maximize the likelihood that the model will generate the

`[reset]` and the safe response when it has already got the input and generated the prefix of an unsafe response.
        - The 2nd part is the standard SFT objective.
    - **DPO**
        - Based on experiment, the paper proposes that the model will not do very well with only SFT (but it's a good warmup).
        - DPO is where the model learns to backtrack by itself.
        - We don't use the groundtruth y^+ as a positive response. Instead, we use:

          $prefix(y^-) \oplus [RESET] \oplus y^+ > y^-$

          This means we want the model to learn the very moment where it should perform the action of backtrack: prefix of unsafe response generated, append a `[reset]` token, and continue the good response.
        - The negative pair, on the other hand, is the y^- itself.
    - **Inference**
        - During the inference time, the API will check if `[reset]` token appears in the sequence. If no, just return the sequence; otherwise, the response after the last `[reset]` token will be returned.
- Experiment Results
    - Significantly enhanced security without compromising capability: On Gemma-2-2B and Llama-3-8B, the SFT+DPO backtracking approach substantially reduced security violation rates (e.g., Llama-3-8B's violation rate dropped from 5.3% to 1.5%, a relative decrease of 72%). Concurrently, MT-Bench scores indicate no decline in model utility.
    - Enhanced resistance to adversarial attacks: Despite lacking specialized training against escape attacks, the backtracking model demonstrated significantly superior security performance compared to baseline models when facing adversarial attacks such as GCG, AutoDAN, and prompt prefilling. Authors even designed an adaptive attack specifically aimed at preventing the model from generating `[reset]`, yet the model still exhibited strong resilience.
    - Controllable trade-off between efficiency and security: Backtracking does increase latency and reduce throughput by generating discarded tokens. However, developers can apply a negative logit bias (e.g., -5) to the `[reset]` token during inference. This maintains nearly equivalent generation efficiency while preserving high security.
- Work Cited
    - Paper
    - Chat History
    - NotebookLM